TxnSails: Achieving Serializable Transaction Scheduling with Self-Adaptive Isolation Level Selection

Qiyu Zhuang
Renmin University of China
qyzhuang@ruc.edu.cn

Wei Lu‡
Renmin University of China
lu-wei@ruc.edu.cn

Shuang Liu
Renmin University of China
shuang.liu@ruc.edu.cn

Yuxing Chen
Tencent Inc.
axingguchen@tencent.com

Xinyue Shi
Renmin University of China
xinyueshi@ruc.edu.cn

Zhanhao Zhao
Renmin University of China
zhanhaozhao@ruc.edu.cn

Yipeng Sun
Renmin University of China
yipengsun@ruc.edu.cn

Anqun Pan
Tencent Inc.
aaronpan@tencent.com

Xiaoyong Du
Renmin University of China
duyong@ruc.edu.cn

ABSTRACT

Achieving the serializable isolation level is costly. Recent studies have revealed that adjusting specific query patterns within the workload can still achieve serializability, even at lower isolation levels. Nevertheless, these studies typically overlook the trade-off between the performance advantages of lower isolation levels and the overhead required to maintain serializability, potentially leading to suboptimal isolation level choices that fail to maximize performance. In this paper, we present TxnSails, a middle-tier solution designed to achieve serializable scheduling with self-adaptive isolation level selection. First, TxnSails incorporates a unified concurrency control algorithm that achieves serializability at lower isolation levels with minimal overhead. Second, TxnSails employs a deep learning method to characterize the trade-off between the performance benefits and overhead associated with lower isolation levels, thus predicting the optimal isolation level. Finally, TxnSails implements a cross-isolation validation mechanism to ensure serializability during real-time isolation level transitions. Extensive experiments demonstrate that TxnSails outperforms existing solutions by up to 26.7× and PostgreSQL 's serializable isolation level by up to 4.8×.

PVLDB Reference Format:
Qiyu Zhuang, Wei Lu, Shuang Liu, Yuxing Chen, Xinyue Shi, Zhanhao Zhao, Yipeng Sun, Anqun Pan, and Xiaoyong Du. TxnSails: Achieving Serializable Transaction Scheduling with Self-Adaptive Isolation Level Selection. PVLDB, 18(11): 4227 - 4240, 2025.
doi:10.14778/3749646.3749689

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/dbiir/TxnSailsServer.

‡Wei Lu is the corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 11 ISSN 2150-8097.
doi:10.14778/3749646.3749689

1 INTRODUCTION

Serializable isolation level (SER) is regarded as the gold standard for transaction processing due to its ability to prevent all forms of anomalies. SER is essential in mission-critical applications, such as banking systems in finance , which require their data to be 100% correct [30, 34, 54]. However, it incurs expensive coordination overhead by configuring the RDBMS to SER [19, 46, 64]. Despite significant efforts to alleviate this overhead [39, 45, 56, 58], maintaining a serial order of transactions to be scheduled remains a fundamental performance bottleneck.

Many studies have explored achieving SER by modifying applications while configuring the RDBMS to a low isolation level [31]. This approach is driven by two key reasons. First, some RDBMSs, such as Oracle 21c, cannot strictly guarantee SER and do not support the in-RDBMS modification, requiring application logic modifications to enforce it [11]. Second, RDBMSs typically offer better performance at lower isolation levels, such as read committed (RC) and snapshot isolation (SI), due to their more relaxed ordering requirements. Modifying applications to achieve SER while using a lower isolation level sometimes results in better performance compared to directly setting the RDBMS to SER [9, 10, 46].

The main idea of existing works that can achieve SER under low isolation levels follows three steps: ❶ Build a *static dependency graph* by analyzing the transaction templates, which are the abstraction of transaction logics in real-world applications [46, 47]. In this graph, each template is represented by a vertex, and the dependencies between templates, such as write-write (WW), write-read (WR), or read-write (RW), are depicted as edges. ❷ Configure the RDBMS to a low isolation level and identify *dangerous structures* that are permissible under the low isolation level but prohibited by SER. For example, under RC, a single RW dependency constitutes the dangerous structure [10, 46]. ❸ Eliminate dangerous structures by modifying application logic, e.g., promoting reads to writes for certain SQL statements so that the RW dependencies are eliminated, and thus guarantees SER. To achieve this, existing works assume that application logic can be abstracted into transaction templates and that all runtime transactions conform

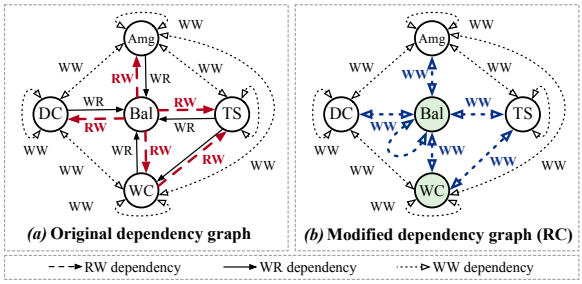


Figure 1: Static dependency graphs of Smallbank

to these templates. This is reasonable as modern applications typically use Object-Relational Mapping (ORM) frameworks [53] to generate structured and repeatable transaction patterns. Therefore, we adopt the same assumption as existing works in this paper, and for interactive or ad hoc transactions not captured by templates, we configure the database to SER to ensure correctness.

Example 1. *Consider the SmallBank benchmark [9], which consists of five transaction templates. As outlined in step ❶, the benchmark is modeled into a static dependency graph (Figure 1a). At step ❷, five dangerous structures (highlighted by red dashed arrows) are identified, including the dependency from WC to TS and 4 dependencies from Bal to the other four templates. At step ❸, extra writes are introduced to convert RW dependencies to WW dependencies, thus eliminating the dangerous structures. To achieve this, certain "SELECT" statements are modified to "SELECT ... FOR UPDATE". A detailed illustration is provided in [1].For reference, the complete modified dependency graph is shown in Figure 1b.* □

Thus far, existing studies [12, 13, 18, 24, 25] have proposed promising solutions, enabling RDBMSs to achieve SER by operating at lower isolation levels while modifying specific query patterns within a workload. However, these studies exhibit two major shortcomings. First, the static modification of query patterns is inefficient. These studies alter static SQL statements at the application level, converting certain read operations into write operations. This may result in unnecessary transaction conflicts. For instance, changing read operations to write operations may turn concurrent read-write operations into write-write conflicts in MVCC systems, thus significantly degrading transaction performance. Second, these studies fail to address the key trade-off between the performance gains of lower isolation levels and the overhead needed to maintain SER, making it difficult to choose the optimal isolation level. As shown in Figure 6 of §7, simply configuring the RDBMS to SER can sometimes outperform other methods. Additionally, as workloads evolve, the ideal isolation level may also shift, but existing studies lack the ability to adapt dynamically.

In this paper, we present TxnSails to address the aforementioned shortcomings with three key objectives: ❶ TxnSails efficiently achieves SER under various low isolation levels. ❷ TxnSails dynamically adjusts the optimal isolation level to maximize performance as the workload evolves. ❸ TxnSails is designed to be general and adaptable across various RDBMSs, requiring no modifications to database kernels. To achieve this, TxnSails is implemented as a middle-tier solution to enhance generalizability. However, implementing TxnSails presents three major challenges. First, designing an approach that elevates various isolation levels to SER without introducing additional writes is a complex task. Second, determining the optimal isolation level requires accurately modeling the trade-offs between the performance benefits and serializability overhead associated with lower isolation levels, which is particularly challenging in dynamic workloads. Third, as workloads evolve, the optimal isolation level may need to adapt over time, making it essential to design an efficient and reliable mechanism for transitioning between isolation levels. To address these challenges, we propose the following key techniques.

(1) Efficient middle-tier concurrency control algorithm ensuring SER for each low isolation level (§4.1). We propose a runtime, fine-grained approach that operates on individual transactions rather than transaction templates, ensuring that the execution of transactions meets the requirements of SER. This approach is inspired by the theorem that a schedule is serializable if it does not contain two transactions, T_i and T_j, where T_j commits before T_i, but there is a dependency from T_i to T_j [10]. Building on this theorem, we introduce a unified concurrency control algorithm to ensure SER. The algorithm tracks transactions with their templates involved in specific RW dependency within a static dependency graph. It detects the runtime dependencies and schedules the commit order to align with their dependency order. If necessary, it will abort a transaction to guarantee SER.

(2) Self-adaptive isolation level selection mechanism (§4.2). We observe that the performance benefits of various isolation levels and the overhead of achieving SER are closely influenced by two critical factors: the data access dependencies between transactions and the data access distribution within transactions. To capture these, we model workload features as a graph, where vertices represent individual transaction features and edges denote dependency features between transactions. Building on this insight, we propose a graph-based model [16, 29] that predicts the optimal isolation level using real-time workload features. To the best of our knowledge, *TxnSails is the first work to enable self-adaptive isolation level selection for dynamic workloads.*

(3) Cross-isolation validation mechanism that enables efficient transitions and serializable scheduling (§4.3). The optimal isolation level should adapt as the workload evolves. When the RDBMS decides to change the isolation level, new transactions must be executed under this updated isolation level. Although existing approaches can achieve SER when all transactions use a unified low isolation level, they fail to ensure SER when transactions operate under different isolation levels. This is because varying isolation levels can introduce new dangerous structures. To address this issue, we identify the structures across different isolation levels and propose a cross-isolation validation mechanism that can guarantee SER during transitions without causing significant system downtime. We prove the correctness of the cross-isolation validation mechanism in §5.2.

We have conducted extensive evaluations on SmallBank [9], TPC-C [5], and YCSB+T [20] benchmarks. The results show that TxnSails achieves up to a 26.7× performance improvement over state-of-the-art methods and up to a 4.8× performance boost compared to SER provided by PostgreSQL.

2 PRELIMINARIES

RDBMSs typically offer several isolation levels; in this paper, we focus on the three most commonly used: serializable (SER), snapshot isolation (SI), and read committed (RC). In this section, we first discuss transaction templates. We then present the dangerous structures under SI and RC, respectively. We finally define the vulnerable dependencies that build the foundation of our approach.

2.1 Transaction Templates

A transaction template is an abstraction of application logic that consists of predefined SQL statements with parameter placeholders. Take the Amalgamate template in Example 1 as an example, which facilitates the transfer of funds from one customer to another. It first reads the balances of the checking and savings accounts of customer N_1, then sets them to zero. Finally, it increases the checking balance for N_2 by the sum of N_1's previous balances. In this context, N_1 and N_2 serve as parameter placeholders. This modular structure ensures readability and flexibility, allowing the transaction template to be reused across various contexts.

For better clarity, we use $\mathcal{T}_i$ to denote a transaction template and T_i to denote a transaction generated by $\mathcal{T}_i$.

2.2 Dangerous Structures

The dependencies between two concurrent transactions, T_i and T_j, operating on the same item x, are classified as follows.

- $T_i \xrightarrow{ww} T_j$: T_i writes a version of data item x, and T_j writes a later version of x.
- $T_i \xrightarrow{wr} T_j$: T_i writes a version of data item x, and T_j reads either the version written by T_i or a later version of x.
- $T_i \xrightarrow{rw} T_j$: T_i reads a version of data item x, and T_j writes a later version of x.

Definition 1 (SI dangerous structure [43]). *Under SI, two consecutive RW dependencies: $T_i \xrightarrow{rw} T_j \xrightarrow{rw} T_k$ are considered as an SI dangerous structure, where T_i and T_j, T_j and T_k are concurrent transactions, respectively.* □

Definition 2 (RC dangerous structure [10, 27]). *Under RC, an RW dependency: $T_i \xrightarrow{rw} T_j$ is considered as an RC dangerous structure, where T_i and T_j are concurrent transactions.* □

When it comes to transaction templates, the dependencies between two transaction templates, $\mathcal{T}_i$ and $\mathcal{T}_j$, are defined as follows: (1) $\mathcal{T}_i \xrightarrow{ww} \mathcal{T}_j$ if $\mathcal{T}_i$ and $\mathcal{T}_j$ write the same data set (e.g., relation) in sequence; (2) $\mathcal{T}_i \xrightarrow{wr} \mathcal{T}_j$ if $\mathcal{T}_i$ writes and $\mathcal{T}_j$ reads the same data set in sequence; (3) $\mathcal{T}_i \xrightarrow{rw} \mathcal{T}_j$ if $\mathcal{T}_i$ reads and $\mathcal{T}_j$ writes the same data set in sequence.

Definition 3 (Static SI dangerous structure [8, 17]). *In a static dependency graph, two consecutive edges $\mathcal{T}_i \xrightarrow{rw} \mathcal{T}_j, \mathcal{T}_j \xrightarrow{rw} \mathcal{T}_k$ are deemed to constitute a static SI dangerous structure.* □

Definition 4 (Static RC dangerous structure [10, 47]). *In a static dependency graph, an edge $\mathcal{T}_i \xrightarrow{rw} \mathcal{T}_j$ is deemed to constitute a static RC dangerous structure.* □

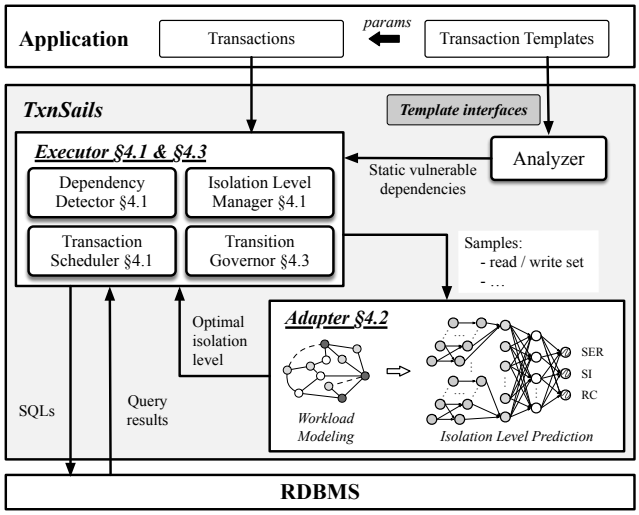


Figure 2: An overview of TxnSails

Theorem 2.1 ([7]). *If a static dependency graph contains no SI (resp. RC) static dangerous structures, then scheduling the transactions generated by the corresponding transaction templates achieves SER when the RDBMS is configured to SI (resp. RC).* □

Theorem 2.1 serves as the foundation for existing approaches to achieving SER while the RDBMS is configured to SI/RC. However, these approaches are static and coarse-grained, leading to the incorrect identification of many non-cyclic schedules. This, in turn, causes a significant number of unnecessary transaction rollbacks.

2.3 Vulnerable Dependency

Definition 5 (Static vulnerable dependency). *The static vulnerable dependency is defined as $\mathcal{T}_j \xrightarrow{rw} \mathcal{T}_k$ in chain $\mathcal{T}_i \xrightarrow{rw} \mathcal{T}_j \xrightarrow{rw} \mathcal{T}_k$ under SI, and $\mathcal{T}_i \xrightarrow{rw} \mathcal{T}_j$ under RC, respectively.* □

Definition 6 (Vulnerable dependency). *The vulnerable dependency is defined as $T_j \xrightarrow{rw} T_k$ in chain $T_i \xrightarrow{rw} T_j \xrightarrow{rw} T_k$ under SI, and $T_i \xrightarrow{rw} T_j$ under RC, respectively.* □

Theorem 2.2 ([10]). *For any vulnerable dependency $T_i \xrightarrow{rw} T_j$, if T_i commits before T_j, then the scheduling achieves SER.* □

Theorem 2.2 forms the basis of our dynamic, fine-grained approach to achieving serializable scheduling. Compared to existing approaches, our approach neither introduces unnecessary writes nor misjudges cyclic schedules, thus preventing unwarranted transaction rollbacks.

3 OVERVIEW OF TxnSails

TxnSails works in the middle tier between the application tier and the database tier, designed to ❶ ensure SER when transactions operate under a low isolation level without introducing additional writes; ❷ select the optimal isolation level for dynamic workloads adaptively; ❸ constantly keep SER during the isolation level transition. An overview of TxnSails is depicted in Figure 2. It comprises three main components: *Analyzer*, *Executor*, and *Adapter*.

Analyzer. *Analyzer* provides *template interfaces* for template registration and analysis. Before TxnSails executes any transaction from the application, *Analyzer* builds the static dependency graph for the transaction templates and identifies all the static vulnerable dependencies for each low isolation level according to Definition 5. It then sends the static vulnerable dependencies to *Executor*.

Executor. *Executor* ensures SER when transactions operate either at a single low isolation level or during the isolation level transition. There are four core modules: *Isolation Level Manager (ILM)*, *Dependency Detector (DD)*, *Transaction Scheduler (TS)*, and *Transition Governor (TG)*. (1) ILM stores the static vulnerable dependencies. Before any transaction T starts, it identifies whether T involves any static vulnerable dependencies. If not, *Executor* sends T directly to the RDBMS for execution; otherwise, ILM triggers DD that identifies vulnerable dependencies of T. (2) DD monitors the reads and writes of T, detecting any runtime vulnerable dependencies between T and other transactions. If T is involved in any vulnerable dependencies, TS is triggered. (3) TS attempts to ensure that the commit and vulnerable dependency orders remain consistent between T and other transactions. If the consistency cannot be guaranteed, T is blocked or aborted; otherwise, T proceeds to commit. (4) TG ensures SER during the transition between two isolation levels. It follows a new corollary, which extends Theorem 2.2 to any two transactions, T_i and T_j, executing under different isolation levels. The proof of correctness during the isolation level transition is detailed in §5.2.

Adapter. *Adapter* models the trade-off between performance benefits and serializability overhead under lower isolation levels. It predicts the optimal isolation level when the workloads evolve. Initially, a dedicated thread is introduced to continuously sample aborted/committed transactions using Monte Carlo sampling [63], capturing the read/write data items. After collecting a batch of transaction samples, *Adapter* predicts the optimal isolation level for future workloads based on the characteristics of the batch. The prediction process consists of two steps: *Workload Modeling (WM)* and *Isolation Level Prediction (ILP)*. WM extracts performance-related features and models the workload as a graph. In this graph, each vertex represents a runtime transaction, with its features capturing the transaction context, such as the number of data items in the read and write sets. Each edge represents an RW or WW operation dependency between transactions. Following WM, ILP embeds the workload graph into a high-dimension vector using graph neural network [16] and message passing techniques [29], and then translates the vector into three possible labels: RC, SI, or SER. The label with the highest value, as determined by our model, indicates the most efficient isolation level.

4 DESIGN OF TxnSails

In this section, we provide the detailed design of TxnSails. We first introduce the middle-tier concurrency control mechanism that achieves serializable scheduling when the RDBMS is configured to a low isolation level (§4.1). Then, we present a self-adaptive isolation level selection approach, which can predict the optimal future isolation level (§4.2). Lastly, we introduce the cross-isolation validation mechanism that ensures serializable scheduling during the isolation level transition (§4.3).

4.1 Middle-tier Concurrency Control

Existing approaches ensure serializability at low isolation levels by statically introducing additional write operations. However, these approaches reduce concurrency and increase overhead. To overcome these limitations, TxnSails introduces a middle-tier concurrency control algorithm, which dynamically validates runtime dependencies and schedules their commit order. In particular, TxnSails focuses exclusively on vulnerable dependencies identified by the *Analyzer* and employs a lightweight validation mechanism to further mitigate overhead.

4.1.1 Transaction lifecycle. The lifecycle of transactions in the middle tier is divided into three phases: execution, validation, and commit phases. (1) In the execution phase, TxnSails establishes a database connection with a specific isolation level, which is not adjusted until the transaction is committed or aborted. Following the RDBMS transaction execution, TxnSails stores the read/write data items in the thread-local buffer that may induce the vulnerable dependencies; (2) In the validation phase, TxnSails acquires validation locks for data items stored in the buffer. Then, it detects the dependencies among them and aims to schedule the commit order consistent with the identified dependency order. A more detailed description of the validation phase will be given in §4.1.2; (3) In the commit phase, TxnSails applies modifications to the database and subsequently releases the validation locks.

4.1.2 Validation phase. TxnSails performs two key tasks in the validation phase: (1) detecting vulnerable dependencies; (2) scheduling the commit order consistent with the dependency order. To achieve this, we utilize a *version* column, which is incremented after every write operation. We trace the dependency orders by comparing the versions of data items. It is worth noting that the explicit version column is introduced for the convenience of description. It can be achieved by extracting version information from the database without modifying the schema. For example, PostgreSQL maintains the version information via the ***ctid***. Due to space constraints, we provide a concise overview of the validation algorithm, while leaving the detailed pseudocode in our technique report [1].

For both RC and SI levels, we detect vulnerable dependencies based on those defined in Definition 6. During validation, the transaction first requests *Shared* locks for items in the read set and *Exclusive* locks for items in the write set. Specifically, before transaction T_i commits, the validation phase is performed in two key steps:

(1) T_i checks each data item in its write set to detect the RW dependencies from T_i's concurrent transactions (e.g., T_j) to T_i. We achieve this via the validation locks. If any lock request fails, indicating T_j exists, an RW dependency is detected. In such a case, the failed lock request should be appended to the corresponding lock's waitlist, making T_i wait until T_j commits, ensuring consistency between dependency and commit orders. If no concurrent transactions in the validation phase are reading the same item, T_i proceeds to commit and create a new data version.

(2) T_i checks each data item in its read sets to detect the RW dependencies from T_i to T_i's concurrent transactions. Specifically, it compares the version of each read item in the thread-local buffer with the latest version in the database. If a newer version is found, indicating an RW dependency from the current transaction T_i to

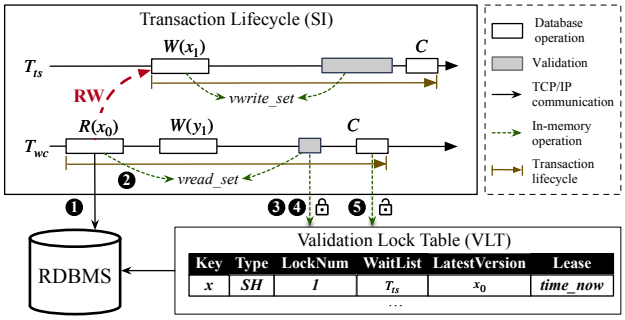


Figure 3: Transaction processing in TxnSails

a committed transaction, say T_j, then T_i is aborted to ensure the consistency of commit and dependency orders. Moreover, comparing data item versions in the local buffer with the latest versions in the RDBMS can introduce additional interactions between the database middleware and the underlying database, imposing overhead on system and network resources. To alleviate this burden, TxnSails employs a caching mechanism in the middle-tier memory to store the latest versions of data items, which can significantly reduce validation overhead.

In the above steps, we ensure the commit order in the middle tier is consistent with the dependency order. Subsequently, we schedule the actual commit order in the RDBMS consistent with the commit order in the middle tier. We ensure that the RDBMS layer consistency is achieved by releasing validation locks only after the transaction has been completed in the RDBMS. Based on this, if two concurrent transactions conflict, they cannot enter the validation phase simultaneously. One transaction must complete validation and commit before the other can proceed, ensuring a correct and consistent commit order in the RDBMS.

To enable efficient and accurate validation, TxnSails leverages a validation lock table (VLT) to maintain metadata for each data item. Each data item is assigned a hash value computed using the collision-resistant hash function $\mathcal{H}$, and a corresponding entry is stored in VLT. Data items with the same hash value are stored in the same bucket and organized as a linked list. When an entry with key x is accessed, TxnSails first determines the appropriate bucket using $\mathcal{H}(x)$ and then traverses it to locate the specific entry. Each hash entry e comprises five fields: (1) $e.Type$, the type of locks acquired, which can be *None*, *Shared (SH)*, and *Exclusive (EX)*; (2) $e.LockNum$, the number of currently held locks; (3) $e.WaitList$, a list of transactions waiting to acquire locks; (4) $e.LatestVersion$, the most recent committed version of the data item; and (5) $e.Lease$, the timestamp indicating the garbage collection time.

Example 2. *Take the transaction processing in Figure 3 as an example. Recall that there exists a static vulnerable dependency $\mathcal{T}_{wc} \xrightarrow{rw} \mathcal{T}_{ts}$ in Smallbank when the RDBMS is set to SI (Figure 1). Thus, it is necessary to detect the read operation of T_{wc} and the write operation of T_{ts}. In the execution phase, after the RDBMS execution (❶), T_{wc} stores the data item x in its vread_set and T_{ts} stores x in its vwrite_set (❷). In the validation phase of T_{wc}, it acquires the shared validation lock on x (❸) and retrieves the latest version of x from either VLT or the RDBMS (❹). While in the validation phase of T_{ts}, it requests the exclusive validation lock on x and is blocked until T_{wc} releases the lock. Finally, in the commit phase, T_{wc} releases the validation lock on x (❺). This ensures that the commit order of the two transactions is consistent with the dependency order, thereby guaranteeing SER when they operate under SI.* □

4.1.3 Discussion. To optimize memory usage, TxnSails incorporates an efficient garbage collection algorithm to evict cold entries. More detailed description is provided in our technical report [1]. Moreover, complex queries make it hard for TxnSails to track dependencies. For complex queries on primary keys, TxnSails uses multi-grained validation locks (such as interval or table locks) to detect predicate dependencies, drawing on techniques similar to SIREAD locks in PostgreSQL and gap locks [38]. When TxnSails cannot easily infer target items in complex queries (e.g., with predicates on non-primary keys), it defaults to table-level locks or configures the underlying database to use SER for correctness.

4.2 Self-adaptive Isolation Level Selection

Selecting optimal isolation levels for all transactions in a workload while maintaining SER is challenging, as we need to balance the extra serializability overhead and performance benefits in different isolation levels. To address this, we propose a neural-network-based isolation level prediction approach, which predicts the future optimal isolation level based on the current workload features [41, 55, 61]. Specifically, TxnSails adopts transaction dependency graphs to capture workload features and adopts a graph classification model to perform self-adaptive isolation level selection.

4.2.1 Graph construction. To extract the complex features of concurrent transactions, TxnSails proposes a graph-structured workload model, which is composed of three matrixes: a vertex matrix V, an edge index matrix E, and an edge attribute matrix A. Formally, a workload graph is defined as $G = (V, E, A)$, where each row in A represents the feature vector of an operator, each entry e_{ij} in E signifies the relationship between v_i and v_j, and each row in A represents the feature vector of an edge.

TxnSails constructs the runtime workload graph by sampling transactions adhering to Monte Carlo sampling. Each transaction in the batch is mapped to a vertex v_i, and its feature vector V_i is generated by extracting the number of data items in its read and write set. For each vertex pair (v_i, v_j), if a data dependency exists between them, i.e., their read and write sets intersect, TxnSails adds an edge entry e_{ij} into the edge index matrix E. For each edge e_{ij}, TxnSails extracts the data dependency type and the involved relations to generate its attribute $A_{e_{ij}}$. The data dependency type for e_{ij} can be either RR, RW/WR, or WW. Given that the number of relations and dependency types is fixed, one-hot encoding is employed to represent these features within the attribute matrix.

4.2.2 Graph embedding and isolation prediction. Predicting the optimal isolation strategy for the future workload using the constructed graph-structure model $G = (V, E, A)$ is challenging due to its complex structures and dynamic and high-dimensional features, which require capturing both local and global dependencies. Heuristic methods rely on manually crafted rules that lack generalizability, while traditional machine learning models are deficient in leveraging relational information encoded in the vertexes

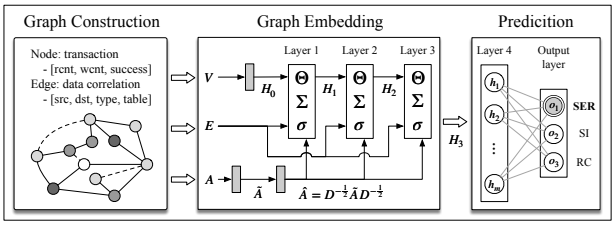


Figure 4: Graph-based isolation level selection model

and edges, losing critical structural context. To address these challenges, we use a graph classification model that learns graph-level representations by aggregating node features through multiple layers of neural network-based convolutions.

As shown in Figure 4, our graph model comprises two parts. First, we use a *Graph Embedding Network* to learn and aggregate both vertex and edge features, producing node-level embedded matrix H that encodes the local structure and attribute information of the graph. Second, to predict the optimal isolation strategy for the workload, we use a *Graph Classification Network* that learns the mapping from the embedded matrix H to perform the end-to-end graph classification to predict the optimal isolation strategy.

The *Graph Embedding Network* is constructed with a three-layer architecture, where each layer applies a convolution operation to update node representations. This process integrates node and edge features through a dynamic aggregation mechanism [26, 62]. At each layer, an edge network maps the input edge features into higher-dimensional convolution kernels via a multi-layer perception (MLP), as shown in Eq.(1). This mapping dynamically transforms edge attributes into weights, which are then used during the node aggregation step. The convolution operation produces updated node embeddings for each node v_i using Eq.(1), where $\mathcal{N}(v_i)$ represents the neighbors of node v_i, $W_{e_{ij}}^{(l)}$ is the edge-specific weight, and σ denotes the active function (i.e., ReLU). Through this layer-wise propagation, the embedding module produces H, a set of node-level embeddings that encode the graph information.

$$\begin{cases} W^{(l)} = f^{(l)}(A) = MLP(A) \\ H_{v_i}^{(l)} = \sigma\left(\max\limits_{v_j \in \mathcal{N}(v_i)} \left(W_{e_{ij}}^{(l)} \cdot H_{v_j}^{(l-1)}\right)\right) \end{cases} \tag{1}$$

The *Graph Classification Network* takes the node embeddings H as inputs and passes them through two fully connected layers. The first layer applies a ReLU activation function to enhance nonlinearity. The second layer implements a softmax activation function and outputs a three-dimensional vector, with each field representing the probability of the isolation level being optimal.

4.2.3 Data collection and labeling. Our modeling approach is somewhat general and not specifically designed for any particular workloads. However, in practice, we train the model separately for each type of benchmark for efficiency considerations. Taking YCSB+T as an example, we generate lots of random workloads with varying read/write ratios and key distributions. Each workload is executed under each isolation level for 10 seconds, with sampling intervals of 1 second, and the optimal isolation level is labeled based on throughput.

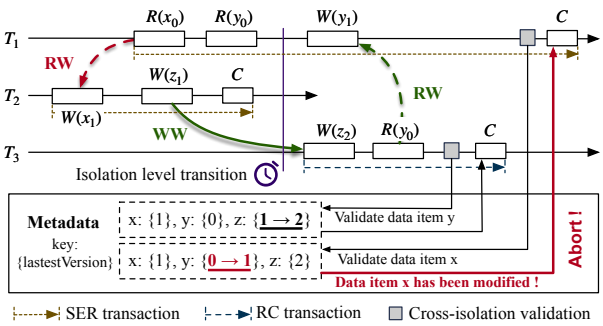


Figure 5: Cross-isolation validation

4.2.4 Model training. In TxnSails, we train the embedding and prediction network together and use cross-entropy loss. Backpropagation involves calculating the gradients of the loss function concerning the parameters of the graph model. First, the gradient is computed for the output layer. Then, using the chain rule, these gradients are propagated backward through the whole network, updating the parameters of each layer. For embedding layers, this process includes computing gradients for both vertex features and transformation matrices derived from edge attributes. Moreover, the model can be retrained asynchronously, and during the retraining period, it does not affect transaction execution.

4.3 Cross-isolation Validation

If the predicted optimal isolation level changes, TxnSails will adapt from the previous isolation level I_{old} to the optimal isolation level I_{new}. We design a cross-isolation validation mechanism to guarantee SER during the isolation level transition.

Example 3. *Figure 5 illustrates non-serializable scheduling during the transition from SER to RC after T_2 commits, making T_1 and T_2 operate under SER while T_3 operates under RC. In this scenario, T_1 is expected to be aborted to ensure SER. However, existing RDBMSs do not handle dependencies between transactions under different isolation levels, allowing T_1 to commit successfully, leading to non-serializable scheduling. Note that when transactions T_1, T_2, and T_3 are all executed under SER, the concurrency control in RDBMS prevents such non-serializable scheduling.* □

We need to explicitly consider the situations of cross-isolation transitions to ensure the correct transaction execution during the process. A straightforward approach is to wait for all transactions to complete under the previous isolation level before making the transition. In the example above, this would mean blocking T_3 until T_1 commits. However, it can result in prolonged system downtime, especially when there are long-running uncommitted transactions. Another possible approach is to abort these uncommitted transactions and retry them after the transition, which leads to a high abort rate. To mitigate these negative impacts, TxnSails employs a cross-isolation validation (CIV) mechanism that ensures serializability and allows for non-blocking transaction execution without a significant increase in aborts. Specifically, we extend the vulnerable dependency under the single isolation level in Definition 6 to the cross-isolation vulnerable dependency, defined as follows:

Definition 7 (Cross-isolation vulnerable dependency). *The cross-isolation vulnerable dependency is defined as* $T_j \xrightarrow{rw} T_k$ *in the dependency chain* $T_i \xrightarrow{rw} T_j \xrightarrow{rw} T_k$ *where* T_j *commits after the transition starts.* □

Based on Definition 7, we extend Theorem 2.2, which is defined for the single isolation level, to derive Corollary 1 for cross-isolation transitions, and prove its correctness in §5.2.

Corollary 1. *During the transition, if every cross-isolation vulnerable dependency* $T_j \xrightarrow{rw} T_k$ *in a transaction scheduling satisfies that* T_j *commits before* T_k*, this scheduling is serializable.* □

Based on Corollary 1, we implement our CIV mechanism by detecting all cross-isolation vulnerable dependencies during the isolation-level transition and ensuring the consistency of the commit and dependency orders. The CIV mechanism includes three steps. (1) When the system transitions from the current isolation I_{old} to the optimal isolation level I_{new}, the middle tier blocks new transactions from entering the validation phase until all transactions that have entered the validation phase before the transition commit or abort. Importantly, we only block transactions to enter the validation phase. Transactions can execute normally without blocking. (2) After that, the transaction that has completed the execution phase enters the cross-isolation validation phase. During the cross-isolation validation phase, transactions request validation locks according to the stricter locking method of either I_{old} or I_{new} to ensure that all cross-isolation vulnerable dependencies can be detected. For example, when transitioning from SI to RC, the transaction in the cross-isolation validation phase requests validation locks following RC's validation locking method, regardless of whether it is executed under SI or RC. (3) After acquiring validation locks, transaction T_i first detects vulnerable dependencies of its original isolation level. Then, it detects cross-isolation vulnerable dependencies by checking whether a committed transaction modifies its read set (using the same detection method as that in §4.1). If such modifications are detected, T_i is aborted to ensure the consistency of the commit and dependency orders. As shown in Figure 5, TxnSails validates $T_1.vread_set$ (x_0 and y_0) before it can commit. Since T_2 has created a new version x_1 and commits before T_1, TxnSails would abort T_1 to ensure serializability.

Once all transactions executed under I_{old} are committed or aborted, the transition process ends. Then, transactions do not need to undergo the cross-isolation validation.

5 SERIALIZABILITY AND RECOVERY

In this section, we first prove the serializability of TxnSails's scheduling in the single-isolation level and cross-isolation level categories in § 5.1 and § 5.2, respectively. Finally, we present the failure recovery strategy in § 5.3.

5.1 Serializability under Low Isolation Levels

Non-serializable scheduling under each low isolation level accommodates certain specific vulnerable dependencies. According to Theorem 2.2, a necessary condition for non-serializability is the presence of inconsistent dependencies and commit orders among these vulnerable dependencies. The unified middle-tier concurrency control ensures the commit order respects dependency order for transactions with vulnerable dependencies, thereby preserving SER even when the RDBMS operates at lower isolation levels.

5.2 Serializability under Cross-isolation Levels

In this subsection, we prove the serializability during isolation transitions after implementing our CIV mechanism in §4.3. The proof can be reduced into two steps: (1) we first prove the correctness of Corollary 1; (2) then we prove that our CIV mechanism can detect each cross-isolation vulnerable dependency, $T_j \xrightarrow{rw} T_k$, in Definition 7 and ensure the consistency between the commit order and the dependency order between T_j and T_k, thus ensuring serializability during isolation transitions.

Correctness of Corollary 1 We prove the correctness of Corollary 1 by proving the correctness of its contrapositive. Corollary 1 can be formalized using Eq.(2) where ct_{T_j} represents the T_j's commit time and ts denotes the isolation transition's start time.

$$\forall (T_i \xrightarrow{rw} T_j \xrightarrow{rw} T_k \wedge ct_{T_j} > ts \wedge ct_{T_j} < ct_{T_k}) \Rightarrow SER \tag{2}$$

Its contrapositive can be deduced using Eq.(3). For clarity, we use α to denote $T_i \xrightarrow{rw} T_j \xrightarrow{rw} T_k \wedge ct_{T_j} > ts$.

$$\begin{aligned}
\neg SER &\Rightarrow \neg\forall(\alpha \wedge ct_{T_j} < ct_{T_k}) \Rightarrow \exists\neg(\alpha \wedge ct_{T_j} < ct_{T_k}) \\
&\Rightarrow \exists(\neg\alpha \vee (\neg ct_{T_j} < ct_{T_k} \wedge (\alpha \vee \neg\alpha))) \\
&\Rightarrow \exists(\neg\alpha \vee (\neg ct_{T_j} < ct_{T_k} \wedge \neg\alpha) \vee (\neg ct_{T_j} < ct_{T_k} \wedge \alpha)) \\
&\Rightarrow \exists((\neg\alpha \wedge (True \vee \neg ct_{T_j} < ct_{T_k})) \vee (\neg ct_{T_j} < ct_{T_k} \wedge \alpha)) \\
&\Rightarrow \exists(\neg\alpha \vee (\neg ct_{T_j} < ct_{T_k} \wedge \alpha))
\end{aligned} \tag{3}$$

Since any witness of $\neg ct_{T_j} < ct_{T_k} \wedge \alpha$ automatically satisties $\neg\alpha \vee (\neg ct_{T_j} < ct_{T_k} \wedge \alpha)$, the proof of Corollary 1's contrapositive can be reduced to proving $\neg SER \Rightarrow \exists(\neg ct_{T_j} < ct_{T_k} \wedge \alpha)$, which is demonstrated in Proposition 1.

Proposition 1. *If a non-serializable schedule occurs during the isolation transition, there must exist a cross-isolation vulnerable dependency,* $T_i \xrightarrow{rw} T_j \xrightarrow{rw} T_k$*, where* T_j *commits after the transition starts, and* T_k *commits before* T_j*.* □

We then prove the correctness of Proposition 1 based on the following two steps. ❶ If there is a non-serializable scheduling during the transition, there exists at least one anomaly structure defined in Definition 8. ❷ If there is a non-serializable scheduling during the transition, there exists at least one anomaly structure that satisfies T_j commits after the transition starts. Due to space constraints, a detailed proof is available in our technical report [1].

Definition 8 (Anomaly Structure). *The anomaly structure during transition is defined as* $T_i \xrightarrow{rw} T_j \xrightarrow{rw} T_k$*, where* T_j *operates under SER and* T_k *commits before* T_j*.*

5.2.1 Serializability during transition. Then we prove that our CIV mechanism can detect all vulnerable dependencies described in Definition 7, thus ensuring the serializability. In the CIV mechanism, we employ validation for every transaction that is not committed after the transition starts, ensuring it is not involved in a cross-isolation vulnerable dependency and maintaining consistent dependency and commit orders. With this validation, we can ensure that there does not exist any cross-isolation vulnerable dependency during the transition, thus ensuring the serializability.

Table 1: Interfaces of TxnSails

register(template_name, sql)	Register each sql with the template names.
analysis()	Analyze and identify static vulnerable dependencies in low isolation levels.

5.3 Failure Recovery

The system incorporates a robust failure recovery mechanism to ensure data consistency and service availability. For TxnSails failures, the RDBMS first automatically rolls back uncommitted transactions because the connection between the RDBMS and TxnSails is lost. Next, the TxnSails is automatically restarted, reconnected to the RDBMS, and then continues the normal execution. For RDBMS failures, the RDBMS is restarted and recovered to a consistent state using its recovery algorithm (e.g., ARIES [42]). For both failures, both components are restarted, relying on the RDBMS's recovery mechanism to ensure consistency.

6 IMPLEMENTATION

We implement TxnSails from scratch using Java and Python, comprising approximately 6,000 lines of Java code and 500 lines of Python code. It can seamlessly integrate with any RDBMS that offers the isolation levels defined in [6, 43].

Interfaces. Applications interact with TxnSails via predefined interfaces, as detailed in Table 1. Applications first register transaction templates by *register* interface, which parses the sql to extract operation types and relations. Then, *analysis* interface is used to identify static vulnerable dependencies under low isolation levels.

Analyzer. We implement *SDGBuilder* class that takes transaction templates as input and constructs a static dependency graph. The graph is then passed to *CycleFinder* class to detect cycles based on the characteristics defined in Theorem 2.1. Finally, it identifies transaction templates with static vulnerable dependencies and stores the results in a *MetaWorker* instance.

Executor. During execution, TxnSails automatically analyzes runtime transactions and identifies which template transactions belong to. It invokes *SQLRewrite()* function to rewrite queries, selecting the appropriate record version if its template is involved in static vulnerable dependencies. Additionally, we implement a critical data structure, *ValidationMetaTable*, which is initialized before any transactions are received to perform middle-tier validation in both single- and cross-isolation scenarios. Organized as a hash table, each bucket in the table represents a list of *ValidationMeta* entries, including *validation lock*, *latest version*, and *lease* information. A dedicated thread handles garbage collection of expired meta entries by comparing the *lease* with the system's real-time clock. Moreover, we implement the *WAIT-DIE* strategy within the *ValidationMetaTable* to prevent deadlocks.

Adapter. We implement the *TransactionCollector* class to collect the read and write sets for transactions and the *RDGBuilder* class to build the runtime dependency graph. *Adapter* is then implemented with the aid of *torch_geometric*, taking the runtime dependency graph as input and outputting the optimal isolation level. To ensure cross-platform compatibility and efficiency, the Python and Java components communicate via *sockets*.

Integration with Apache Shardingsphere [35]. The *Analyzer* and *Adapter* can be directly integrated as external modules. Specifically, we extend the *PostgreSQLMultiStatementsHandler* to collect validation information from execution results and store it in the *ConnectionSession*, based on static vulnerable dependencies. Additionally, we modify the *TransactionBackendHandler* to perform validation using the metadata gathered during execution, ensuring consistency between dependency and commit order.

7 EVALUATIONS

In this section, we evaluate TxnSails's performance compared to state-of-the-art solutions. Our goal is to validate two critical aspects empirically: (1) TxnSails's effectiveness in adaptively selecting the appropriate isolation level for dynamic workloads (§7.2); and (2) TxnSails's performance superiority over state-of-the-art solutions across a variety of scenarios (§7.3).

7.1 Setup

We run our database and clients on two separate in-cluster servers with an Intel(R) Xeon(R) Platinum 8361HC CPU @ 2.60GHz processor, which includes 24 physical cores, 64 GB DRAM, and 500 GB SSD. The operating system is CentOS Linux release 7.9.

7.1.1 Default configuration. We utilize BenchBase [22] as our benchmark simulator, which is deployed on the client server. By default, the experiments are conducted using 128 client terminals. We deployed PostgreSQL 15.2 [4] as the default database engine, which employs MVCC to implement three distinct isolation levels: RC, SI, SER (by SSI [17]). To prevent dirty writes, write locks are enforced at all isolation levels. For our database configuration, we allocated a buffer pool size of 24GB, limited the maximum number of connections to 2000, and established a lock wait timeout of 100 ms. To eliminate network-related effects, both TxnSails and PostgreSQL were deployed on another server.

7.1.2 Baselines. To ensure a fair comparison, we implemented existing approaches within the BenchBase framework.

Baselines within the database. We evaluated concurrency control algorithms supported natively by RDBMSs, specifically those associated with lower isolation levels that can achieve SER:

*(1) & (2) Native concurrency control mechanisms in RDBMSs (**SER** and **SI**).* These approaches execute workloads configured at the SER or SI levels. For instance, TPC-C achieves serializable scheduling under SI, while SmallBank requires SER for serializability.

Baselines outside the database kernel. We also evaluated external strategies that transform RW dependencies into WW dependencies to eliminate static dangerous structures.

*(3) & (4) Promotion (**RC+Promotion**[47], **SI+Promotion** [9]).* This strategy converts read operations into write operations by promoting *SELECT* statements with non-modifying *UPDATE* statements [9]. These modifications are applied at RC and SI levels, referred to as RC+Promotion and SI+Promotion, respectively.

*(5) & (6) Conflict materialization (**RC+ELM** [10], **SI+ELM** [9]).* This approach employs an external lock manager (ELM) and introduces additional write operations on the ELM to ensure serializable scheduling. It is applied at both RC and SI levels, referred to as RC+ELM and SI+ELM, respectively.

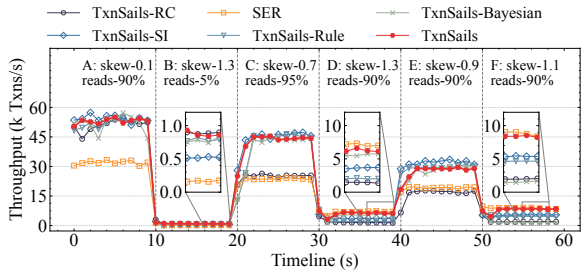


Figure 6: Workload shifting - YCSB

We incorporate two simple isolation level selection methods: a rule-based method and a Bayesian model-based method [28], denoted as **TxnSails-Rule** and **TxnSails-Bayesian**, respectively. TxnSails-Rule relies on the write/read ratio (wr), using SI for $wr < 0.2$, SER for $0.2 \leq wr \leq 0.4$, and RC for $wr > 0.4$. TxnSails-Bayesian uses a 4-dimensional feature vector, including the number of reads (ro), writes (wo), RW dependencies (rw), and WW dependencies (ww), to represent each workload.

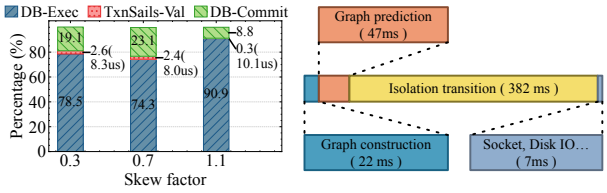
Finally, we evaluate the middle-tier concurrency control in §4.1 at both RC and SI levels without self-adaptive isolation level selection, denoted as **TxnSails-RC** and **TxnSails-SI**, respectively.

7.1.3 Benchmarks. Three benchmarks are conducted as follows.
SmallBank [9]. This benchmark populates the database with 400k accounts, each having associated checking and savings accounts. Transactions are selected by each client using a uniform distribution. To simulate transactional access skew, we employ a Zipfian distribution with a default *skew factor* of 0.7.
YCSB+T [21]. This benchmark generates synthetic workloads emulating large-scale Internet applications. In our setup, the *usertable* consists of 10 million records, each 1KB in size, totaling 10GB. The *skew factor*, set by default to 0.7, controls the distribution of accessed data items, with higher values increasing data contention. Each default transaction involves 10 operations, with a 90% probability of being a read and a 10% probability of being a write.
TPC-C [5]. We use the TPC-C benchmark, which modifies the templates to convert all predicate reads into key-based accesses according to our baseline [46]. It includes five transaction templates: NewOrder, Payment, OrderStatus, Delivery, and StockLevel. Our tests use 32 warehouses, with a total data size of 3.2 GB.

7.2 Ablation Study

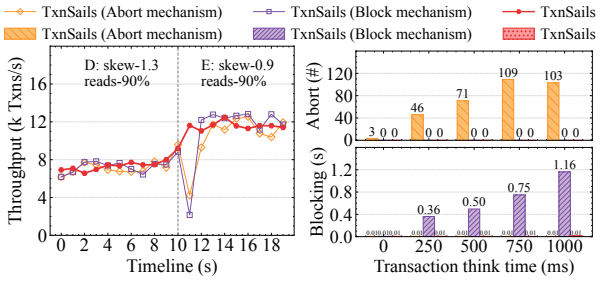
In this part, we evaluate the effectiveness of the self-adaptive isolation level selection and isolation transition in TxnSails.

7.2.1 Self-adaptive isolation level selection. We first evaluate self-adaptive isolation level selection by varying the workload every 10 seconds across six distinct scenarios. The experimental results are illustrated in Figure 6. We sample the workload at 1-second intervals. The results demonstrate that different isolation levels perform variably under different workloads: SI performs well in low-skew scenarios (A, C, E), SER is more effective in high-skew scenarios with a lower percentage of write operations (D, F), and RC excels in high skew scenarios with a high percentage of writes (B). Across all tested dynamic scenarios, TxnSails successfully adapts to the



(a) Transaction breakdown **(b) Transition breakdown**

Figure 7: Breakdown analysis - YCSB

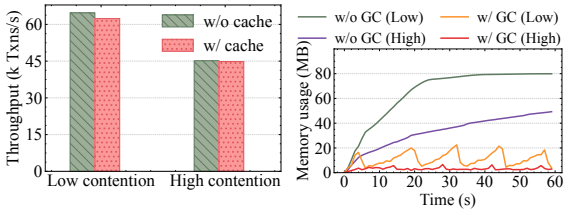


(a) Transition from D to E **(b) Performance metrics**

Figure 8: Comparison of transition mechanisms - YCSB

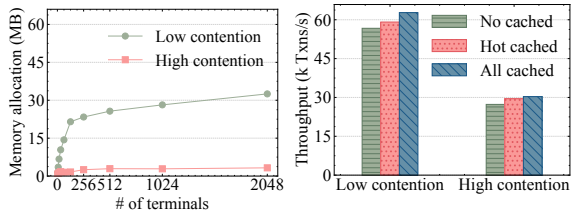
optimal isolation level. Specifically, the graph learning model in TxnSails identifies that SI is suitable for scenarios with fewer conflicts due to its higher concurrency and lower data access overhead. Conversely, RC is ideal for scenarios with higher conflict rates and more write operations, as it efficiently handles concurrent writes (SI aborts concurrent writes, while RC allows them to commit). Compared to an application directly run on RDBMS at the SER level, the performance of TxnSails when selecting SER is slightly reduced by 4.3%. This overhead arises from two aspects: (1) TxnSails requires selecting *version* and another localhost message delivery ; (2) TxnSails needs to sample transactions to predict the optimal isolation level, even though this is an asynchronous task. The rule-based and Bayesian model-based methods achieve correct predictions in some cases; however, they fail to make optimal decisions in scenarios such as workloads D and F, due to their limited modeling capacity compared to the neural network.

7.2.2 Validation analysis. This part evaluates the validation efficiency in both single-isolation and cross-isolation scenarios.
Single-isolation level validation. We evaluate the single-level validation cost under skew factors of 0.3, 0.7, and 1.1, respectively. The average breakdown is depicted in Figure 7a. The validation cost remains relatively stable, decreasing from 2.6% to 0.3% of the transaction lifecycle as contention increases. This suggests that the middle-tier concurrency control proposed in §4.1 does not significantly affect normal execution.
Cross-isolation level validation. We evaluate TxnSails with YCSB using the various transition mechanisms mentioned in §4.3. We first evaluate the transition of the workload from D to E with a "think time" of 1s, as illustrated in Figure 8a. TxnSails minimizes the impact of isolation level transitions by avoiding active block



(a) Performance (b) Memory usage

Figure 9: Impact of garbage collection - YCSB



(a) Memory usage (b) Performance

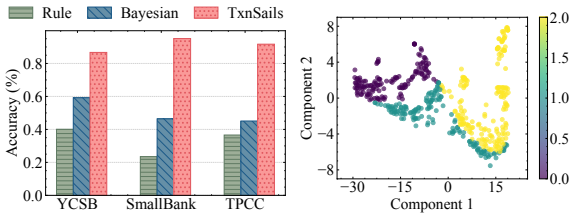
Figure 10: Impact of Metadata - YCSB

time or aborts. To further compare mechanisms, we vary the "think time" parameter (Figure 8b). Increased think time raises transaction latency and leads to more aborts under the abort strategy, and the blocking strategy incurs longer blocking times. In contrast, the cross-isolation validation mechanism outperforms both, reducing transaction aborts and blocking time while delivering performance improvements of up to 2.7× and 5.4×, respectively.

7.2.3 Overhead analysis. We first evaluate the metadata-induced memory overhead, as shown in Figure 10. With 10GB of YCSB data, TxnSails uses under 50MB of memory while achieving a 10% performance gain. The cache hit ratio is 39.8% under low contention and 68.4% under high contention, indicating that TxnSails efficiently caches frequently accessed tuples, especially when contention is high. Then, we evaluate the overhead of our garbage collection strategy. As illustrated in Figure 9, the garbage collection mechanism incurs only slight performance overheads, 3.61% under low contention and 0.6% under high contention, while reducing memory usage by 84.7% and 92.0%, respectively.

7.2.4 Graph model: construction, training, and prediction. Figure 7b illustrates the overhead of workload transition, which takes about 450 ms, with graph construction and prediction accounting for 22 ms and 47 ms, respectively. Over 80% of the time is spent on transition, from initiating the transition to all connections adopting the new isolation level, closely tied to the longest transaction execution latency. Notably, the prediction in Figure 6 is inaccurate for 1 or 2 seconds at the 30-second and 50-second marks due to the sampling transactions from the previous workload. However, the model successfully transitions to the optimal isolation level in subsequent prediction cycles. The overhead of the learned model is minimal, incurring under 2.5% throughput overhead.

We compare the accuracy of three prediction methods and show the result in Figure 11a. The neural network method consistently



(a) Accuracy (b) Extracted features

Figure 11: Model training metrics - YCSB

achieves the highest accuracy across all benchmarks, outperforming the rule-based and Bayesian models by up to 27.5%, 48.7%, and 46.6%, respectively. The Bayesian model mispredicts at Workload E due to unstable decisions near boundaries, where small changes in probability affect isolation level choices. The superior performance of the neural network is due to its ability to capture complex relationships in the dependency graph, whereas the other two simpler approaches rely on irreversibly compressed or aggregated features, limiting their effectiveness. To visualize the high-dimensional vectors produced by our model, we use t-SNE [2] for nonlinear dimensionality reduction, mapping them into two dimensions and plotting them with their true labels in Figure 11b. Most workloads are accurately distinguished, with few errors primarily occurring at the boundaries between isolation levels.

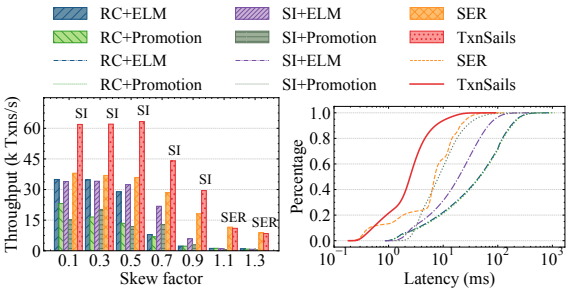
Summary. One isolation level does not fits all workloads. In low-skew scenarios, SI outperforms RC; in high-skew scenarios with fewer writes, SER is the most effective; and in high-skew scenarios with intensive writes, RC is more suitable. TxnSails effectively guarantees SER at lower levels and efficiently adapts isolation levels to optimize performance for dynamic workloads using the proposed fast isolation level transition technique.

7.3 Comparision to State-of-the-art Solutions

We evaluate TxnSails against state-of-the-art solutions that use **external lock manager (ELM)** [9, 10] and **Promotion** [9, 47] over workloads by YCSB, SmallBank, and TPC-C benchmarks.

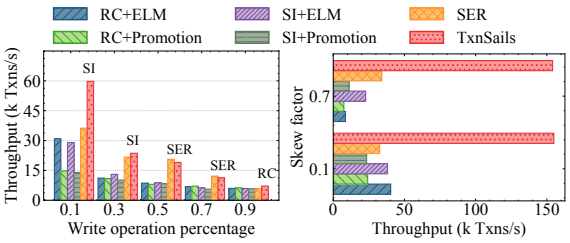
7.3.1 Impact of data contention. This part studies the impact of data contention by varying the *skew factor* (SF). TxnSails outperforms other solutions by up to 22.7× and is 1.9× better than the second-best solution due to lightweight validation without workload modification, thus higher concurrency as depicted in Figure 12a. In this case, SOTA solutions can not beat SER as they introduce additional write operations in YCSB workloads that restrict concurrency. In high contention scenarios (SF>0.9), validation costs outweigh the benefits of using a lower isolation level, triggering TxnSails to transition to the SER level and perform slightly (<5%) lower than SER due to TxnSails overhead. In low contention scenarios (SF<0.9), TxnSails adopts SI due to low validation overhead. We further analyze the latency distribution with the skew factor of 0.9 using cumulative distribution function (CDF) plots, as shown in Figure 12b. In all scenarios, TxnSails reduces transaction latency.

7.3.2 Impact of write/read ratios. This part evaluates the performance of varying the percentage of write operations over YCSB. In



(a) Performance (b) Analysis of latency CDF

Figure 12: Impact of data contention - YCSB



(a) Skew factor is 0.1 (b) Read only transactions

Figure 13: Impact of write/read ratio - YCSB

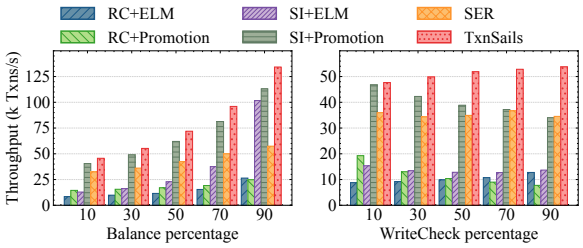


Figure 14: Impact of templates percentage - SmallBank

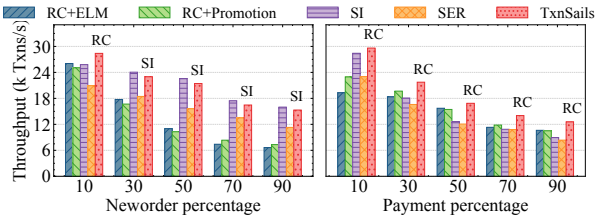


Figure 15: Impact of templates percentage - TPC-C

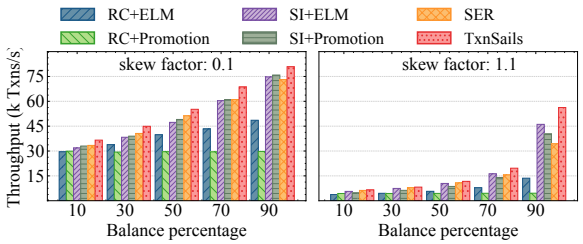


Figure 16: Extended evaluations on MySQL - SmallBank

read-write scenarios in Figure 13a , TxnSails can outperform second best solutions up to 1.7×. As the percentage of write operations increases, the performance gap narrows as verification overhead at lower isolation levels increases. TxnSails transitions from using SI to SER and finally to RC, as the FCW [19] strategy increases the abort rate in scenarios with a high percentage of writes.

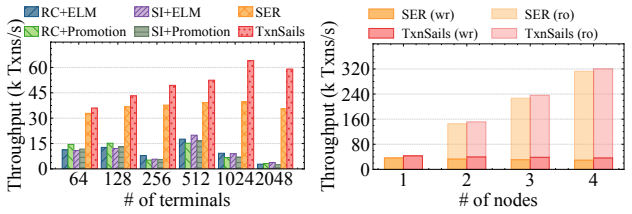
We also evaluate the performance in read-only scenarios in Figure 13b. TxnSails achieves performance up to 4.6× higher than SER and up to 20.4× higher than others. TxnSails adopts to SI level as its in-memory validation is nearly costless and rarely fails. Other solutions convert read operations to write operations, thereby restricting concurrency. This also highlights that when a database is configured to be SER, there is a significant performance loss compared to SI, even with read-only scenarios.

7.3.3 Impact of templates percentages. In complex workloads like SmallBank and TPC-C, only certain transaction templates lead to data anomalies, so modifying these templates can ensure serializability under low isolation levels. This part compares different solutions by varying the percentage of critical transaction templates.

In SmallBank, we evaluate the proportions of the read-only *Balance* and write transaction *WriteCheck*, as shown in Figure 14. As the proportion of *Balance* transactions increases, performance improves; however, RC+ELM and RC+Promotion introduce additional writes in *Balance*, leading to increased WW conflicts. In contrast, SI+ELM and SI+Promotion perform better since they do not modify read-only *Balance* transactions. TxnSails transitions to SI in these workloads and achieves up to 6.2× performance gain over RC+ELM and RC+Promotion. Conversely, as the proportion of *WriteCheck* transactions increases, concurrency decreases. However, TxnSails's performance advantage becomes more pronounced as no extra WW conflicts are introduced. At 90% *WriteCheck* transactions, TxnSails outperforms SI+Promotion by 58.1% and achieves 2.3× the performance of SER.

TPC-C can execute serializable under SI, eliminating the need for validation in SI. The critical *NewOrder* and *Payment* transactions require modifications by RC+ELM and RC+Promotion, increasing write conflicts on *NewOrder*, resulting in a performance disadvantage compared to TxnSails . Due to high contention on the warehouse relation, validation overheads are generally higher, except when the proportion of *NewOrder* is 0.1, where TxnSails shows a 10.7% improvement over SI. In other scenarios, TxnSails adapts to SI. *Payment* transactions are more write-intensive, prompting TxnSails to adapt to RC, which achieves up to 40.6% performance improvement over SI. Compared to other solutions, TxnSails achieves up to 53.5% performance improvement.

7.3.4 Generality and scalability. This part evaluates the generality and scalability of TxnSails using MySQL and Neon [3]. Specifically, we apply TxnSails to low isolation levels in MySQL and conduct experiments using the SmallBank benchmark. As shown



(a) Scale up **(b) Scale out**

Figure 17: Scalability in Neon - YCSB

Table 2: Integerate TxnSails into Apache shardingsphere

Approach	Throughput (k Txn/s)	Median Latency (ms)	p99 Latency (ms)
SSP+TxnSails	32.3	3.97	21.66
SSP	28.4	4.76	49.12

in Figure 16, TxnSails consistently achieves the highest throughput on MySQL. In particular, TxnSails outperforms the second-best solution by up to 12.5% under low contention and 22.1% under high contention workloads. Furthermore, TxnSails surpasses other solutions by up to 12.36×. These results highlight the generality of TxnSails across diverse database systems.

To evaluate the scalability, we conduct experiments on Neon, an elastically scalable database, from both scale-up and scale-out perspectives. As shown in Figure 17a, we vary the number of client terminals on a single read-write node from 32 to 2048. The results show that TxnSails consistently outperforms other solutions. TxnSails achieves up to 1.65× performance improvement at 2048 terminals. As shown in Figure 17b, we increase the number of read-only nodes. Here, TxnSails also consistently surpasses the baseline Neon database, confirming its great scale-out scalability. It is worth noting that, since read-only nodes only provide SI, TxnSails does not add extra concurrency control overhead on these nodes. Consequently, the performance gains become less pronounced as the number of read-only nodes increases.

7.3.5 Integration into Shardingsphere. We evaluate TxnSails in Shardingsphere using the underlying data source PostgreSQL via the YCSB workload with the default configuration; the results are illustrated in Table 2. After integrating TxnSails, SSP's performance improves 13.7%, the median latency reduces 16.5%, and the maximum latency reduces up to 55.9%. TxnSails can provide better performance and lower tail latency with a low isolation level.

Summary. Current research often limits concurrency and scalability in a coarse-grained manner by replacing read locks with write locks. In contrast, TxnSails employs validation-based concurrency control in a fine-grained manner, achieving superior performance compared to existing approaches. Furthermore, unlike previous work that merely advocates for a lower isolation level, we argue that, due to the varying structures and proportions of different transaction templates, higher isolation levels can sometimes yield better results, which can be used by TxnSails adaptively.

8 RELATED WORK

Our study is related to the previous work on concurrency control algorithms that ensure serializable scheduling within and outside the database kernel.

Within the database kernel. Existing works have explored a variety of algorithms to guarantee SER, which can be divided into two main categories: (1) deterministic algorithms (e.g., Calvin [44], Aria [40], Harmony [33]) and (2) non-deterministic algorithms, such as 2PL, OCC, timestamp ordering (TO) and their variants [14, 15, 32, 36, 45, 48, 50–52, 56, 57, 59, 60]. While these algorithms effectively achieve serializable scheduling, they offer varying performance benefits depending on the workload. To address this, some studies propose adaptive concurrency control algorithms for dynamic workloads. For instance, Polyjuice [49] employs reinforcement learning to design tailored concurrency control mechanisms for each stored procedure, while Snapper [37] mixes deterministic and non-deterministic (i.e., 2PL) algorithms. However, deterministic algorithms impose stricter requirements than TxnSails: (1) transactions must be pre-collected and completed in a single interactive round, and (2) their read-write sets must be known in advance. Furthermore, all these algorithms are explicitly tailored for database kernels, which limits their broader applicability and generalizability. In contrast, TxnSails requires no kernel modifications . More importantly, TxnSails boosts performance by adaptively assigning the optimal isolation level based on workload characteristics while preserving SER.

Outside the database kernel. Substantial efforts have been dedicated to scheduling entire workloads under low isolation levels can still achieve SER by adjusting specific query patterns. Fekete et al. provide the necessary and sufficient conditions for SI to achieve serializable scheduling [8, 23]. Ketsman et al. [31, 46] investigate the characteristics of non-serializable scheduling under RC and Read Uncommitted isolation levels. This theoretical framework has been further refined with functional constraints by Vandevoort et al. [47]. Based on these insights, TxnSails can accurately and efficiently achieve SER across various isolation levels.

9 CONCLUSION

In this paper, we present TxnSails, an efficient middle-tier approach that achieves serializability by strategically selecting between serializable and low isolation levels for dynamic workloads. TxnSails introduces a unified middle-tier validation method to enforce the commit order consistent with the vulnerable dependency order, ensuring serializability in single-isolation and cross-isolation scenarios. Moreover, TxnSails adopts a graph-learned model to extract the runtime workload characteristics and adaptively predict the optimal isolation levels, achieving further performance improvement. The experimental results show that TxnSails can self-adaptively select the optimal isolation level and outperform state-of-the-art solutions and the native PostgreSQL.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (No. 62441230, 62472429, 62461146205). Wei Lu is the corresponding author.

REFERENCES

[1] 2024. Supplemental materials of TxnSails. https://github.com/dbiir/TxnSailsServer/blob/main/technique_report_revision.pdf.

[2] 2024. t-distributed stochastic neighbor embedding. https://en.wikipedia.org/wiki/T-distributed_stochastic_neighbor_embedding.

[3] 2025. Neon Serverless Postgres — Ship faster. https://neon.tech/.

[4] 2025. PostgreSQL: The World's Most Advanced Open Source Relational Database. https://www.postgresql.org/.

[5] 2025. TPC-C: On-Line Transaction Processing Benchmark. http://www.tpc.org/tpcc/.

[6] Atul Adya, Barbara Liskov, and Patrick E. O'Neil. 2000. Generalized Isolation Level Definitions. In *ICDE*. IEEE Computer Society, 67–78.

[7] Mohammad Alomari et al. 2009. Ensuring serializable executions with snapshot isolation dbms. (2009).

[8] Mohammad Alomari, Michael Cahill, Alan Fekete, and Uwe Röhm. 2008. Serializable executions with snapshot isolation: Modifying application code or mixing isolation levels?. In *Database Systems for Advanced Applications: 13th International Conference, DASFAA 2008, New Delhi, India, March 19-21, 2008. Proceedings 13*. Springer, 267–281.

[9] Mohammad Alomari, Michael J. Cahill, Alan D. Fekete, and Uwe Röhm. 2008. The Cost of Serializability on Platforms That Use Snapshot Isolation. In *ICDE*. IEEE Computer Society, 576–585.

[10] Mohammad Alomari and Alan D. Fekete. 2015. Serializable use of Read Committed isolation level. In *AICCSA*. IEEE Computer Society, 1–8.

[11] Peter Bailis, Aaron Davidson, Alan D. Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. Highly Available Transactions: Virtues and Limitations. *Proc. VLDB Endow.* 7, 3 (2013), 181–192.

[12] Peter Bailis, Alan D. Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2014. Coordination Avoidance in Database Systems. *Proc. VLDB Endow.* 8, 3 (2014), 185–196.

[13] Peter Bailis, Alan D. Fekete, Joseph M. Hellerstein, Ali Ghodsi, and Ion Stoica. 2014. Scalable atomic visibility with RAMP transactions. In *SIGMOD Conference*. ACM, 27–38.

[14] Claude Barthels, Ingo Müller, Konstantin Taranov, Gustavo Alonso, and Torsten Hoefler. 2019. Strong consistency is not hard to get: Two-Phase Locking and Two-Phase Commit on Thousands of Cores. *Proc. VLDB Endow.* 12, 13 (2019), 2325–2338.

[15] Philip A. Bernstein and Nathan Goodman. 1981. Concurrency Control in Distributed Database Systems. *ACM Comput. Surv.* 13, 2 (1981), 185–221.

[16] Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. 2014. Spectral Networks and Locally Connected Networks on Graphs. In *ICLR*.

[17] Michael J. Cahill, Uwe Röhm, and Alan D. Fekete. 2008. Serializable isolation for snapshot databases. In *SIGMOD Conference*. ACM, 729–738.

[18] Michael J. Cahill, Uwe Röhm, and Alan D. Fekete. 2009. Serializable isolation for snapshot databases. *ACM Trans. Database Syst.* 34, 4 (2009), 20:1–20:42.

[19] Yuxing Chen, Anqun Pan, Hailin Lei, Anda Ye, Shuo Han, Yan Tang, Wei Lu, Yunpeng Chai, Feng Zhang, and Xiaoyong Du. 2024. TDSQL: Tencent Distributed Database System. *Proc. VLDB Endow.* 17, 12 (2024), 3869–3882.

[20] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *SoCC*. ACM, 143–154.

[21] Akon Dey, Alan D. Fekete, Raghunath Nambiar, and Uwe Röhm. 2014. YCSB+T: Benchmarking web-scale transactional databases. In *ICDE Workshops*. IEEE Computer Society, 223–230.

[22] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. 2013. OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proc. VLDB Endow.* 7, 4 (2013), 277–288.

[23] Alan D. Fekete. 2005. Allocating isolation levels to transactions. In *PODS*. ACM, 206–215.

[24] Alan D. Fekete. 2019. Making Consistency Protocols Serializable. In *PODS*. ACM, 269.

[25] Alan D. Fekete, Shirley Goldrei, and Jorge Perez Asenjo. 2009. Quantifying Isolation Anomalies. *Proc. VLDB Endow.* 2, 1 (2009), 467–478.

[26] Satoshi Furutani, Toshiki Shibahara, Mitsuaki Akiyama, Kunio Hato, and Masaki Aida. 2019. Graph Signal Processing for Directed Graphs Based on the Hermitian Laplacian. In *ECML/PKDD (1) (Lecture Notes in Computer Science)*, Vol. 11906. Springer, 447–463.

[27] Yifan Gan, Xueyuan Ren, Drew Ripberger, Spyros Blanas, and Yang Wang. 2020. IsoDiff: Debugging Anomalies Caused by Weak Isolation. *Proc. VLDB Endow.* 13, 11 (2020), 2773–2786.

[28] Andrew Gelman, John B Carlin, Hal S Stern, and Donald B Rubin. 1995. *Bayesian data analysis*. Chapman and Hall/CRC.

[29] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *ICML (Proceedings of Machine Learning Research)*, Vol. 70. PMLR, 1263–1272.

[30] Long Gu, Si Liu, Tiancheng Xing, Hengfeng Wei, Yuxing Chen, and David A. Basin. 2024. IsoVista: Black-box Checking Database Isolation Guarantees. *Proc. VLDB Endow.* 17, 12 (2024), 4325–4328. https://doi.org/10.14778/3685800.3685866

[31] Bas Ketsman, Christoph Koch, Frank Neven, and Brecht Vandevoort. 2022. Deciding Robustness for Lower SQL Isolation Levels. *ACM Trans. Database Syst.* 47, 4 (2022), 13:1–13:41.

[32] Kangnyeon Kim, Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. 2016. ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads. In *SIGMOD Conference*. ACM, 1675–1687.

[33] Ziliang Lai, Chris Liu, and Eric Lo. 2023. When Private Blockchain Meets Deterministic Database. *Proc. ACM Manag. Data* 1, 1 (2023), 98:1–98:28. https://doi.org/10.1145/3588952

[34] Hexu Li, Hengfeng Wei, Hongrong Ouyang, Yuxing Chen, Na Yang, Ruohao Zhang, and Anqun Pan. 2025. Online Timestamp-Based Transactional Isolation Checking of Database Systems. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 3738–3750.

[35] Ruiyuan Li, Liang Zhang, Juan Pan, Junwen Liu, Peng Wang, Nianjun Sun, Shanmin Wang, Chao Chen, Fuqiang Gu, and Songtao Guo. 2022. Apache ShardingSphere: A Holistic and Pluggable Platform for Data Sharding. In *ICDE*. IEEE, 2468–2480.

[36] Hyeontaek Lim, Michael Kaminsky, and David G. Andersen. 2017. Cicada: Dependably Fast Multi-Core In-Memory Transactions. In *SIGMOD Conference*. ACM, 21–35.

[37] Yijian Liu, Li Su, Vivek Shah, Yongluan Zhou, and Marcos Antonio Vaz Salles. 2022. Hybrid Deterministic and Nondeterministic Execution of Transactions in Actor Systems. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*. ACM, 65–78.

[38] David B. Lomet. 1993. Key Range Locking Strategies for Improved Concurrency. In *VLDB*. Morgan Kaufmann, 655–664.

[39] David B. Lomet, Alan D. Fekete, Rui Wang, and Peter Ward. 2012. Multi-version Concurrency via Timestamp Range Conflict Management. In *ICDE*. IEEE Computer Society, 714–725.

[40] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2020. Aria: A Fast and Practical Deterministic OLTP Database. *Proc. VLDB Endow.* 13, 11 (2020), 2047–2060. http://www.vldb.org/pvldb/vol13/p2047-lu.pdf

[41] Lin Ma, Dana Van Aken, Ahmed Hefny, Gustavo Mezerhane, Andrew Pavlo, and Geoffrey J. Gordon. 2018. Query-based Workload Forecasting for Self-Driving Database Management Systems. In *SIGMOD Conference*. ACM, 631–645.

[42] C. Mohan, Don Haderle, Bruce G. Lindsay, Hamid Pirahesh, and Peter M. Schwarz. 1992. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Trans. Database Syst.* 17, 1 (1992), 94–162.

[43] Dan R. K. Ports and Kevin Grittner. 2012. Serializable Snapshot Isolation in PostgreSQL. *Proc. VLDB Endow.* 5, 12 (2012), 1850–1861.

[44] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *SIGMOD Conference*. ACM, 1–12.

[45] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *SOSP*. ACM, 18–32.

[46] Brecht Vandevoort, Bas Ketsman, Christoph Koch, and Frank Neven. 2021. Robustness against Read Committed for Transaction Templates. *Proc. VLDB Endow.* 14, 11 (2021), 2141–2153.

[47] Brecht Vandevoort, Bas Ketsman, Christoph Koch, and Frank Neven. 2022. Robustness Against Read Committed for Transaction Templates with Functional Constraints. In *ICDT (LIPIcs)*, Vol. 220. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 16:1–16:17.

[48] Donghui Wang, Yuxing Chen, Chengyao Jiang, Anqun Pan, Wei Jiang, Songli Wang, Hailin Lei, Chong Zhu, Lixiong Zheng, Wei Lu, Yunpeng Chai, Feng Zhang, and Xiaoyong Du. 2025. TXSQL: Lock Optimizations Towards High Contented Workloads. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 675–688. https://doi.org/10.1145/3722212.3724457

[49] Jia-Chen Wang, Ding Ding, Huan Wang, Conrad Christensen, Zhaoguo Wang, Haibo Chen, and Jinyang Li. 2021. Polyjuice: High-Performance Transactions via Learned Concurrency Control. In *15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2021, July 14-16, 2021*. 198–216.

[50] Tianzheng Wang, Ryan Johnson, Alan Fekete, and Ippokratis Pandis. 2015. The Serial Safety Net: Efficient Concurrency Control on Modern Hardware. In *Proceedings of the 11th International Workshop on Data Management on New Hardware* (Melbourne, VIC, Australia). Association for Computing Machinery, New York, NY, USA, Article 8, 8 pages.

[51] Tianzheng Wang, Ryan Johnson, Alan D. Fekete, and Ippokratis Pandis. 2017. Efficiently making (almost) any concurrency control mechanism serializable. *VLDB J.* 26, 4 (2017), 537–562.

[52] Tianzheng Wang, Ryan Johnson, Alan D. Fekete, and Ippokratis Pandis. 2018. Erratum to: Efficiently making (almost) any concurrency control mechanism serializable. *VLDB J.* 27, 6 (2018), 899–900.

[53] Zhaoguo Wang, Chuzhe Tang, Xiaodong Zhang, Qianmian Yu, Binyu Zang, Haibing Guan, and Haibo Chen. 2024. Ad Hoc Transactions through the Looking Glass: An Empirical Study of Application-Level Transactions in Web Applications. *ACM Trans. Database Syst.* 49, 1 (2024), 3:1–3:43.

[54] Hengfeng Wei, Jiang Xiao, Na Yang, Si Liu, Zijing Yin, Yuxing Chen, and Anqun Pan. 2025. Boosting End-to-End Database Isolation Checking via Mini-Transactions. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 3998–4010.

[55] Tao Yu, Zhaonian Zou, Weihua Sun, and Yu Yan. 2024. Refactoring Index Tuning Process with Benefit Estimation. *Proc. VLDB Endow.* 17, 7 (2024), 1528–1541.

[56] Xiangyao Yu, Andrew Pavlo, Daniel Sánchez, and Srinivas Devadas. 2016. TicToc: Time Traveling Optimistic Concurrency Control. In *SIGMOD Conference*. ACM, 1629–1642.

[57] Xiangyao Yu, Yu Xia, Andrew Pavlo, Daniel Sánchez, Larry Rudolph, and Srinivas Devadas. 2018. Sundial: Harmonizing Concurrency Control and Caching in a Distributed OLTP Database Management System. *Proc. VLDB Endow.* 11, 10 (2018), 1289–1302.

[58] Chao Zhang, Guoliang Li, Leyao Liu, Tao Lv, and Ju Fan. 2025. CloudyBench: A Testbed for A Comprehensive Evaluation of Cloud-Native Databases. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2535–2547.

[59] Chao Zhang, Guoliang Li, and Tao Lv. 2024. HyBench: A New Benchmark for HTAP Databases. *Proc. VLDB Endow.* 17, 5 (2024), 939–951. https://doi.org/10.14778/3641204.3641206

[60] Zhanhao Zhao, Hongyao Zhao, Qiyu Zhuang, Wei Lu, Haixiang Li, Meihui Zhang, Anqun Pan, and Xiaoyong Du. 2023. Efficiently Supporting Multi-Level Serializability in Decentralized Database Systems. *IEEE Trans. Knowl. Data Eng.* 35, 12 (2023), 12618–12633.

[61] Qiushi Zheng, Zhanhao Zhao, Wei Lu, Chang Yao, Yuxing Chen, Anqun Pan, and Xiaoyong Du. 2024. Lion: Minimizing Distributed Transactions Through Adaptive Replica Provision. In *ICDE*. IEEE, 2012–2025.

[62] Xuanhe Zhou, Ji Sun, Guoliang Li, and Jianhua Feng. 2020. Query Performance Prediction for Concurrent Queries using Graph Embedding. *Proc. VLDB Endow.* 13, 9 (2020), 1416–1428.

[63] Zeheng Zhou, Ying Jiang, Weifeng Liu, Ruifan Wu, Zerong Li, and Wenchao Guan. 2024. A Fast Algorithm for Estimating Two-Dimensional Sample Entropy Based on an Upper Confidence Bound and Monte Carlo Sampling. *Entropy* 26, 2 (2024), 155.

[64] Qiyu Zhuang, Xinyue Shi, Shuang Liu, Wei Lu, Zhanhao Zhao, Yuxing Chen, Tong Li, Anqun Pan, and Xiaoyong Du. 2025. GeoTP: Latency-Aware Geo-Distributed Transaction Processing in Database Middlewares . In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 433–445.